Georgia State University

CSC 4520/6520
Design & Analysis of Algorithms

CHAPTER 6: TRANSFORM-AND-CONQUER (AVL)

Abdullah Bal, PhD

Objectives

Recap

Transform-and-Conquer

Instance Simplification

Presorting

Gaussian Elimination

Searching Problem

Binary Search Trees

AVL Trees

Searching Problem

- Problem: Given a (multi)set *S* of keys and a search key *K*, find an occurrence of *K* in *S*, if any

- Searching must be considered in the context of:
  - file size
  - dynamics of data (static vs. dynamic)

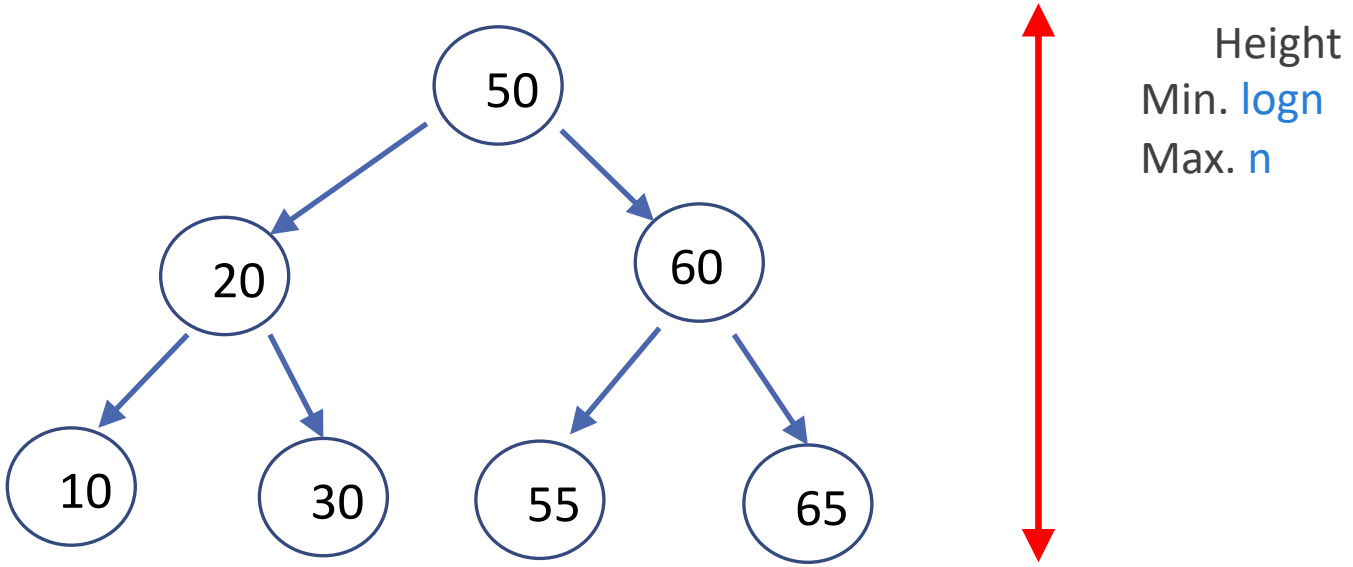
- Dictionary operations (dynamic data):
  - find (search)
  - insert
  - delete

Taxonomy of Searching Algorithms

- List searching
  - sequential search
  - binary search
  - interpolation search
- Tree searching
  - binary search tree
  - binary balanced trees: AVL trees, red-black trees
  - multiway balanced trees: 2-3 trees, 2-3-4 trees, B trees
- Hashing
  - open hashing (separate chaining)
  - closed hashing (open addressing)

Binary Search Trees

- Binary search tree is a binary tree whose nodes contain elements of a set of orderable items,
  - One element per node,
  - All elements in the left subtree are smaller than the element in the subtree's root,
  - All the elements in the right subtree are greater than it.

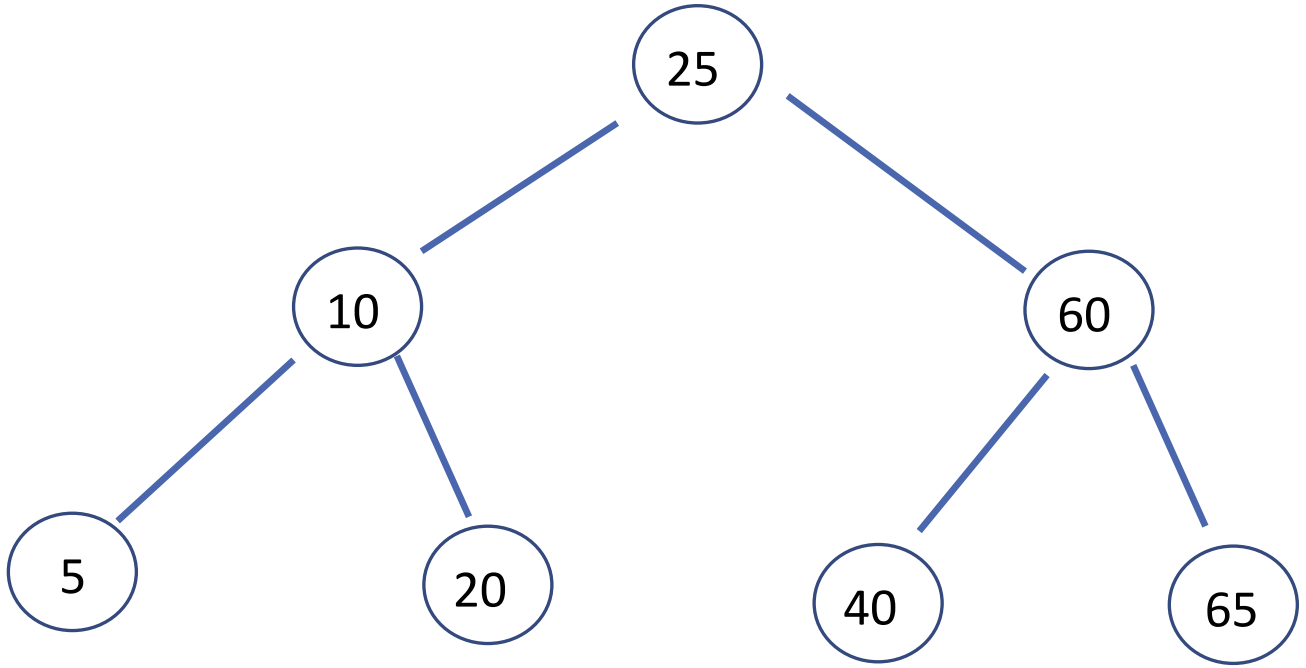


Example: Binary Search Tree

Keys: 25, 60, 10, 65, 20, 5, 40

Example: Binary Search Tree

Keys: 25, 60, 10, 65, 20, 5, 40



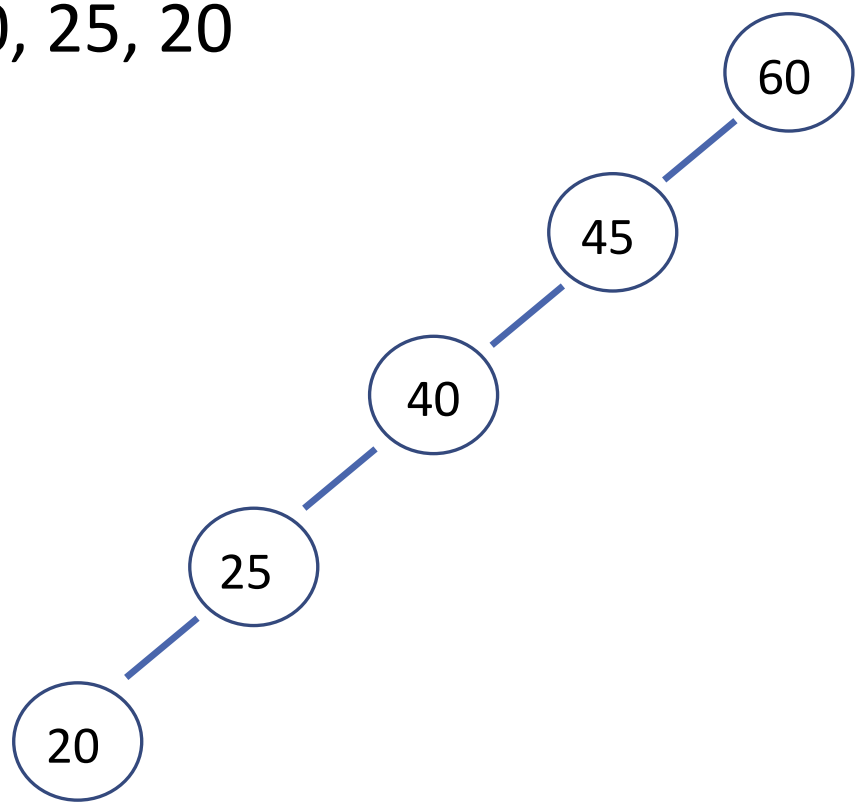
Height:
logn

Example: Binary Search Tree

Keys: 60, 45, 40, 25, 20, 10, 5

Example: Binary Search Tree

Keys: 60, 45, 40, 25, 20



Height

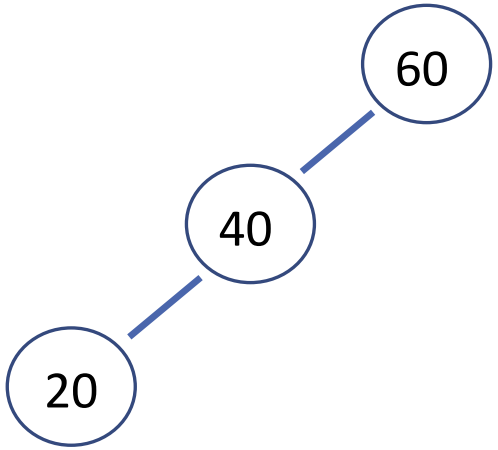
n

Binary Search Tree Variations

What are the variations of [60,40,20]?

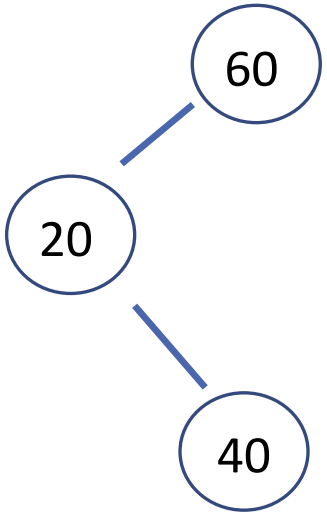
Binary Search Tree Variations

Keys: 60,40,20



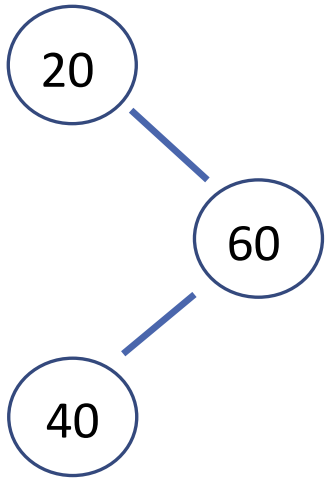
Binary Search Tree Variations

Keys: 60,20,40



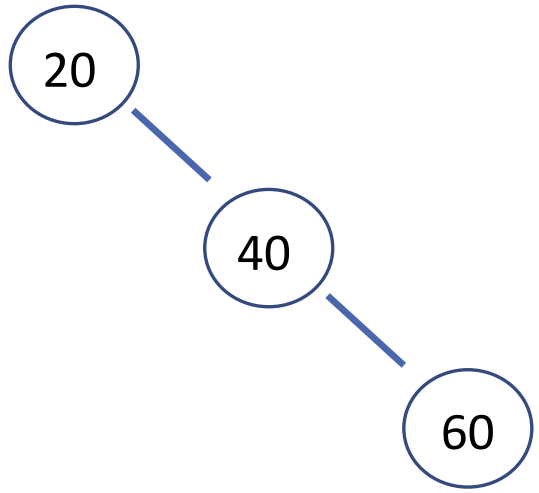
Binary Search Tree Variations

Keys: 20,60,40



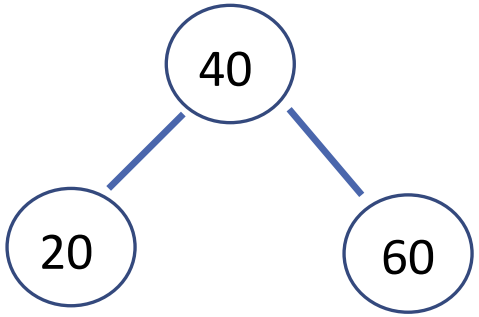
Binary Search Tree Variations

Keys: 20,40,60



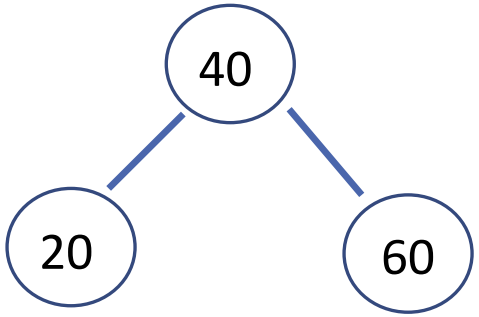
Binary Search Tree Variations

Keys: 40,20,60



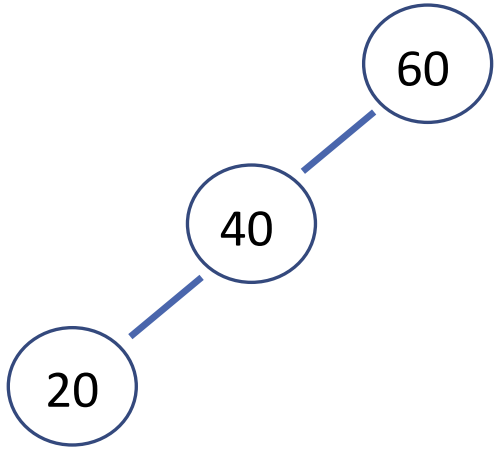
Binary Search Tree Variations

Keys:
40,60,20

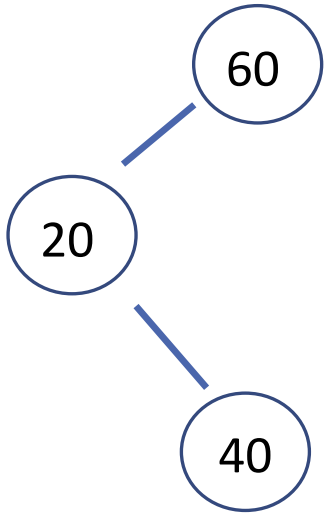


Binary Search Tree Variations

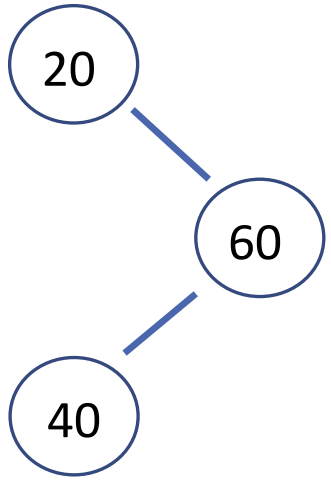
Keys: 60,40,20



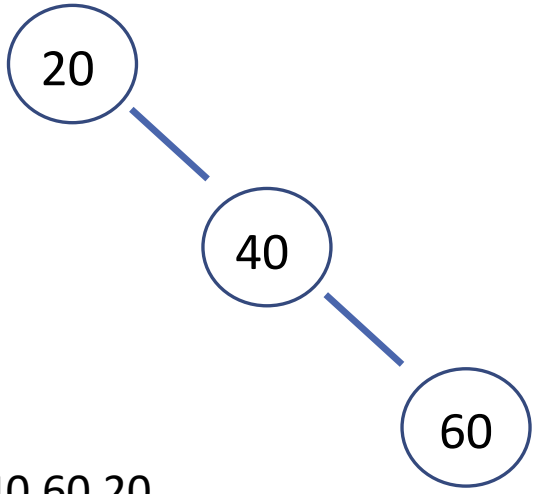
Keys: 60,20,40



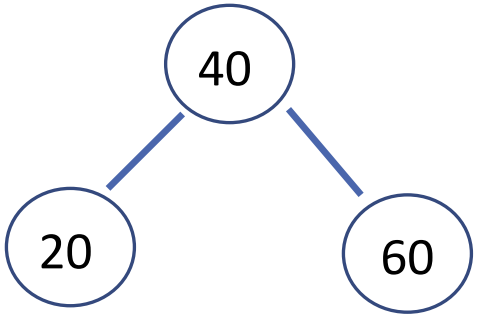
Keys: 20,60,40



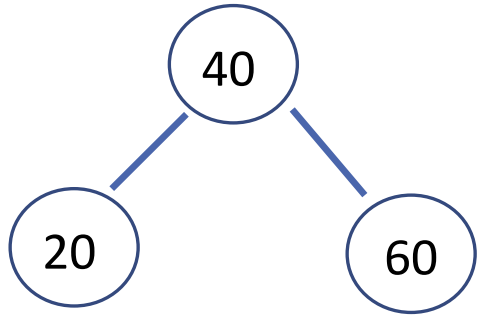
Keys: 20,40,60



Keys: 40,20,60



Keys: 40,60,20



Binary Search Trees

- This transformation from a set to a binary search tree is an example of the representation-change technique.
- We gain in the time efficiency of searching, insertion, and deletion, which are all in $\Theta(\log n)$, but only in the average case.
- In the worst case, these operations are in $\Theta(n)$ because the tree can degenerate into a severely unbalanced one with its height equal to $n - 1$.

Binary Search Trees

- Computer scientists have expended a lot of effort in trying to find a structure that preserves the good properties of the classical binary search tree
- Principally, the logarithmic efficiency of the dictionary operations avoiding its worst-case degeneracy.
- They have come up with two approaches:
  - Instance-simplification variety
  - Representation-change variety

SELF-BALANCING AVL TREE

The Instance-simplification Variety (AVL)

- An unbalanced binary search tree is transformed into a balanced one.
- Such trees are called ***self-balancing***.
- An ***AVL tree*** requires the difference between the heights of the left and right subtrees of every node never exceed 1.
- If an insertion or deletion of a new node creates a tree with a violated balance requirement, the tree is restructured by one of a family of special transformations called ***rotations*** that restore the balance required.

The Representation-change Variety (2-3 Trees)

- Allow more than one element in a node of a search tree.
- Specific cases of such trees are ***2-3 trees***, ***2-3-4 trees***, and more general and important ***B-trees***.
- They differ in the number of elements admissible in a single node of a search tree, but all are perfectly balanced.
- We discuss the simplest case of such trees, the 2-3 tree, in this section, leaving the discussion of *B*-trees for Chapter 7.

Balanced Search Trees

- Attractiveness of binary search tree is marred by the worst-case efficiency. Two ideas to overcome it are:
- To rebalance binary search tree when a new insertion makes the tree "too unbalanced"
  - AVL trees
  - Red-black trees
- To allow more than one key per node of a search tree
  - 2-3 trees
  - 2-3-4 trees
  - B-trees

Balanced trees: AVL

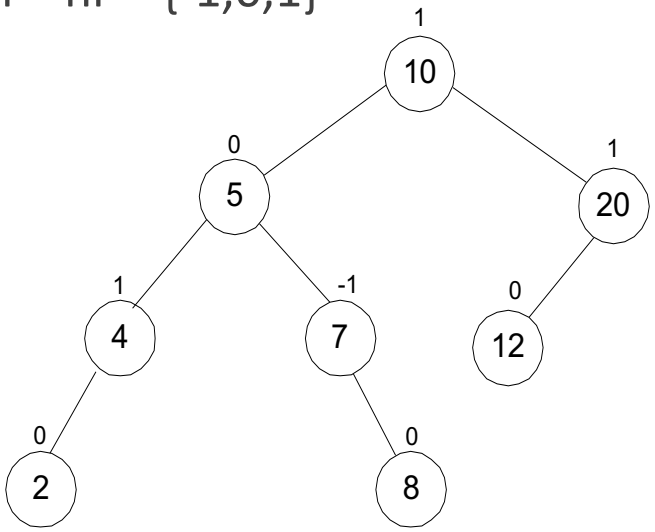
- AVL (**A**delson-**V**elsky-**L**andis) trees were invented in 1962 by two Russian scientists, G. M. Adelson-Velsky and E. M. Landis, after whom this data structure is named.

AVL trees

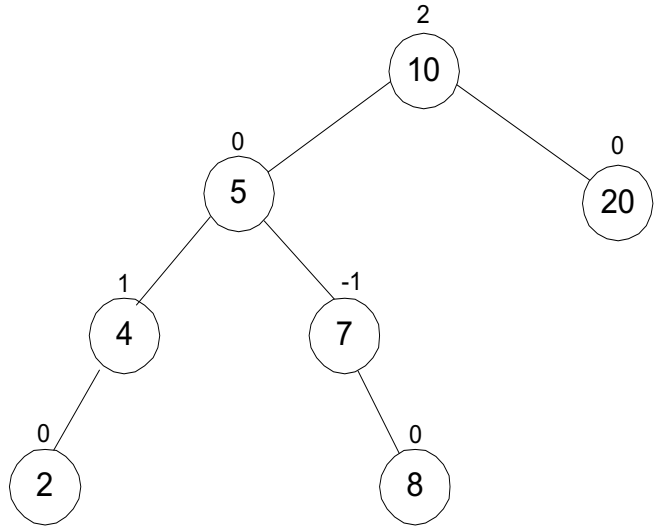
- **An *AVL tree* is a binary search tree** in which the ***balance factor*** of every node, which is defined as the difference between the heights of the node's left and right subtrees, is either 0 or +1 or −1.

balance factor = height of the left subtree – height of the right subtree

balance factor = hl – hr = {-1,0,1}



AVL Tree



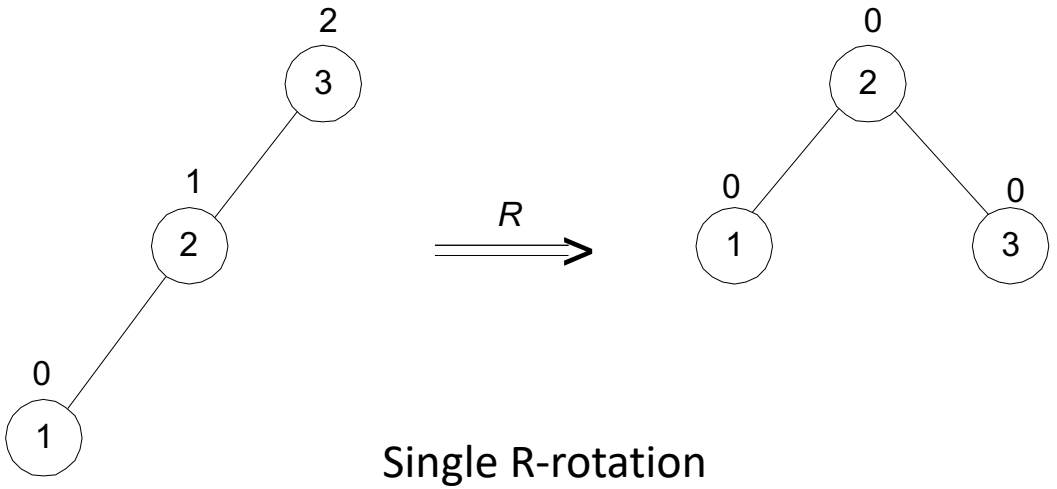
Not AVL Tree

Rotations

- If an insertion of a new node makes an AVL tree unbalanced, we transform the tree by a rotation.
- If there are several such nodes, we rotate the tree rooted at the unbalanced node that is the closest to the newly inserted leaf.
- There are only four types of rotations; in fact, two of them are mirror images of the other two.

Single Right Rotation (R-Rotation)

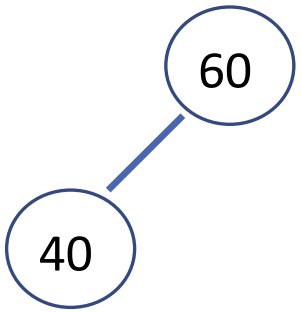
- The first rotation type is called the ***single right rotation***, or ***R-rotation***.
- Note that this rotation is performed after a new key is inserted into the left subtree of the left child of a tree whose root had the balance of +1 before the insertion.



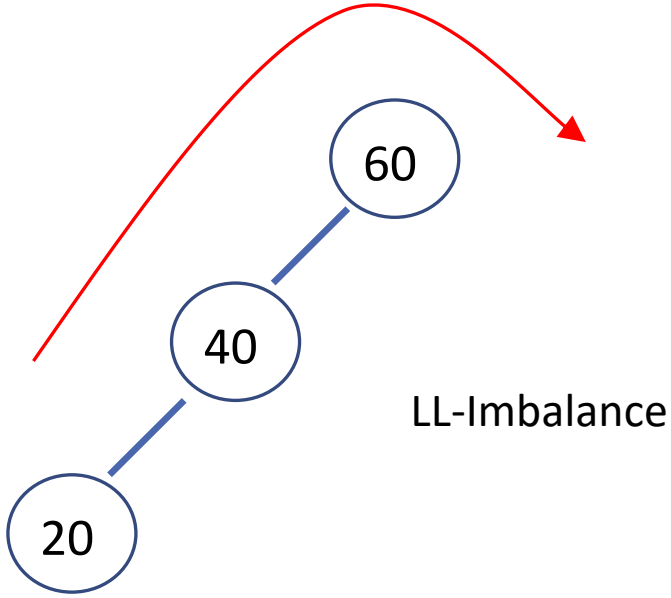
Single R-rotation

Example: Single R Rotation

Keys: 60,40

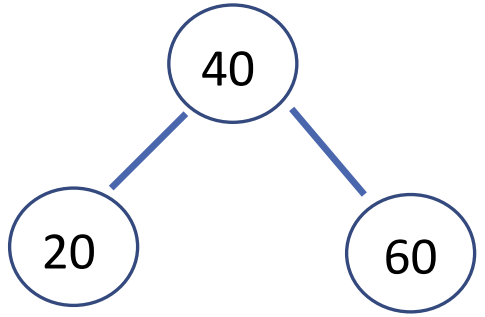


Insert: 20



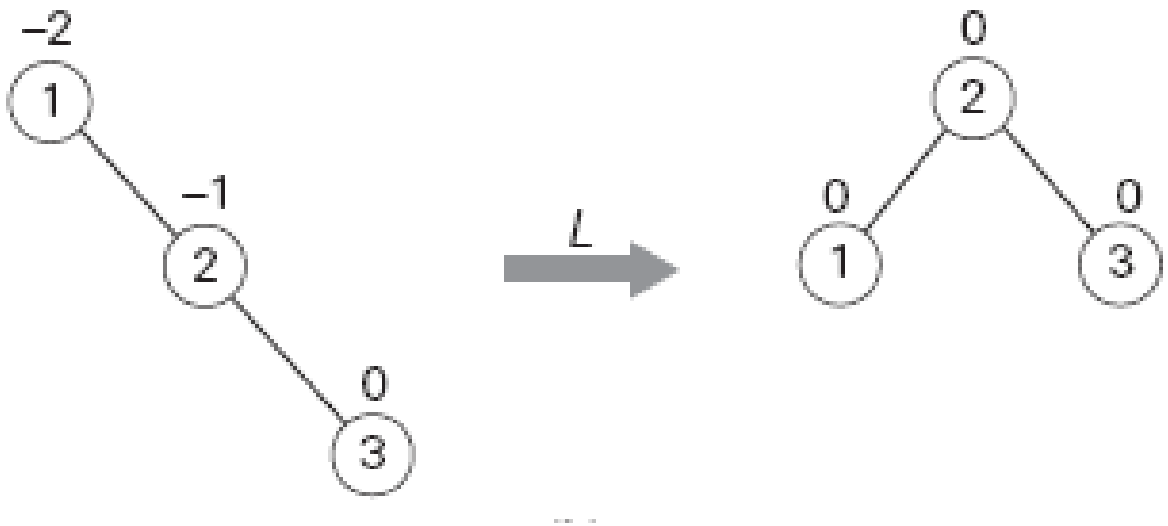
LL-Imbalance

R- Rotation



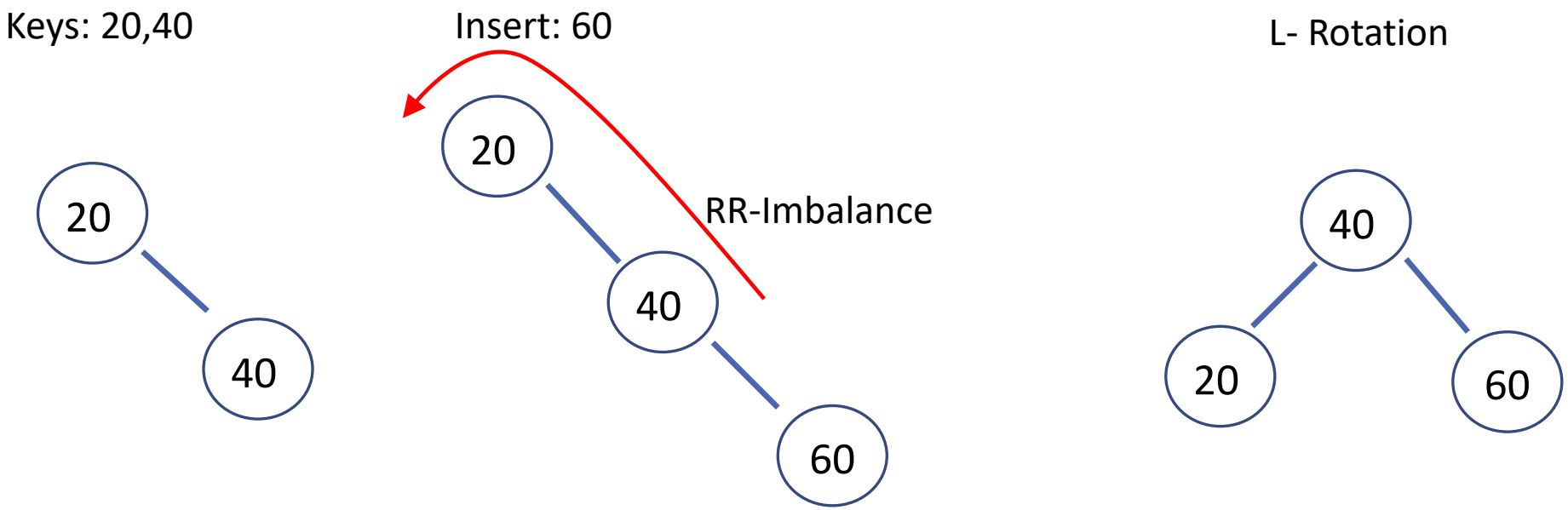
Single Left Rotation

- The symmetric ***single left rotation***, or ***L-rotation***, is the mirror image of the single R-rotation.
- It is performed after a new key is inserted into the right subtree of the right child of a tree whose root had the balance of −1 before the insertion.



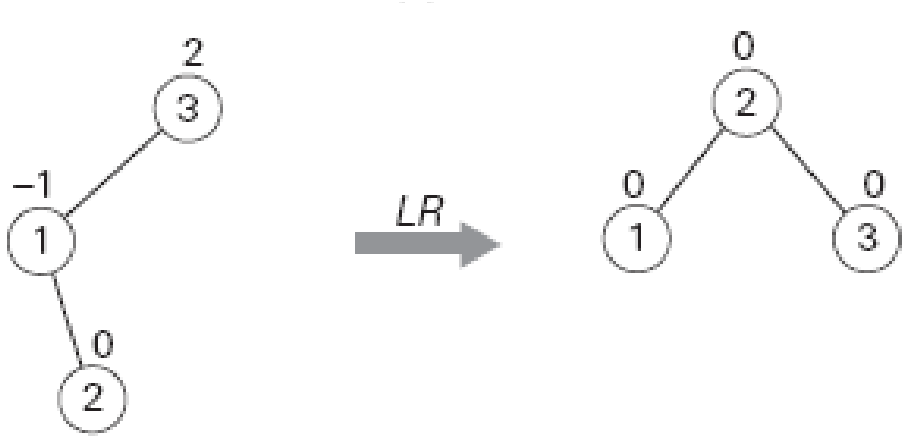
Single L-rotation

Example: Single L Rotation



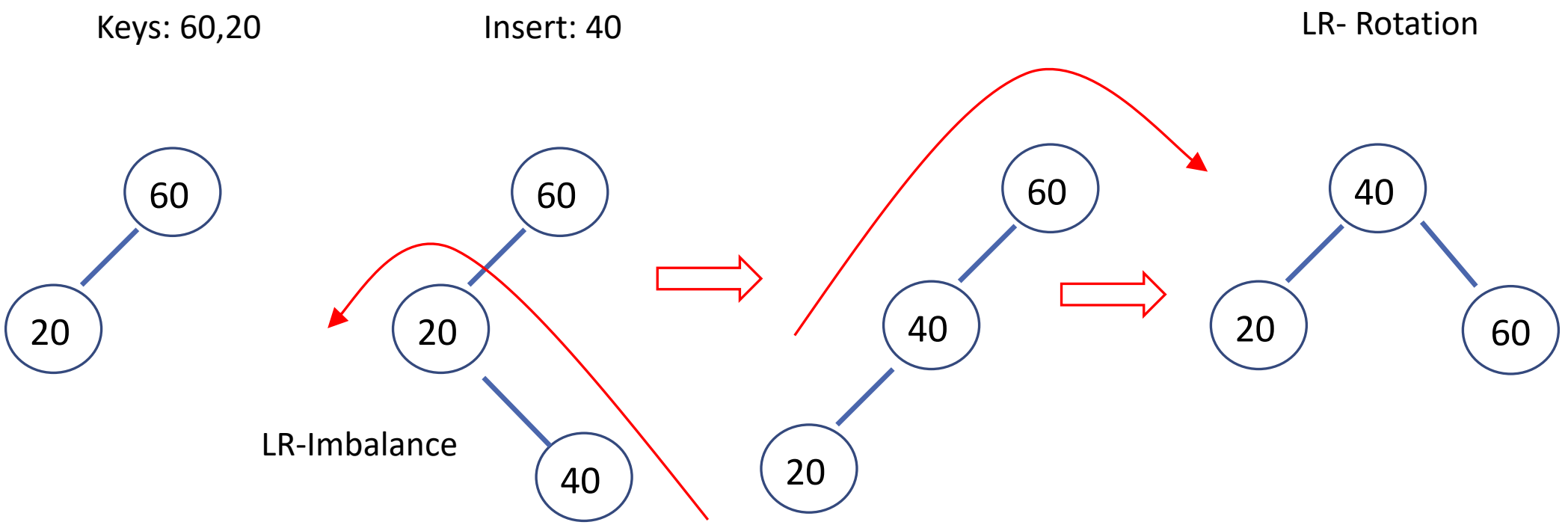
Double Left-Right (LR) Rotation

- The second rotation type is called the ***double left-right rotation*** (***LR- rotation***).
- We perform the *L*-rotation of the left subtree of root r followed by the *R*-rotation of the new tree rooted at r.
- It is performed after a new key is inserted into the right subtree of the left child of a tree whose root had the balance of +1 before the insertion.



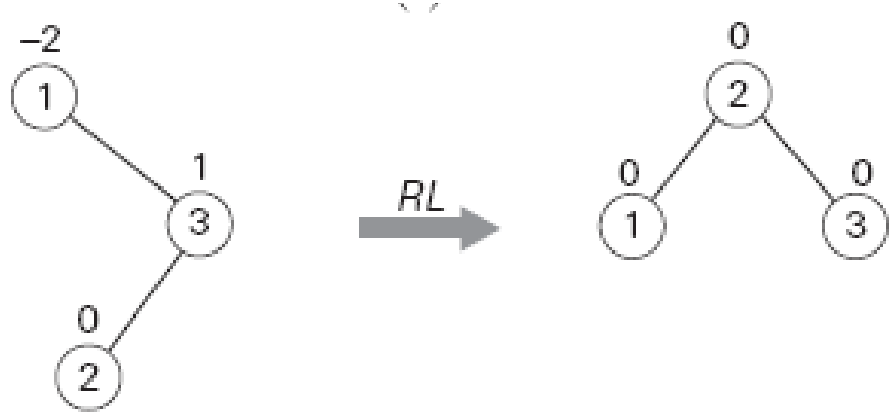
Double LR-rotation

Example: Double LR Rotation



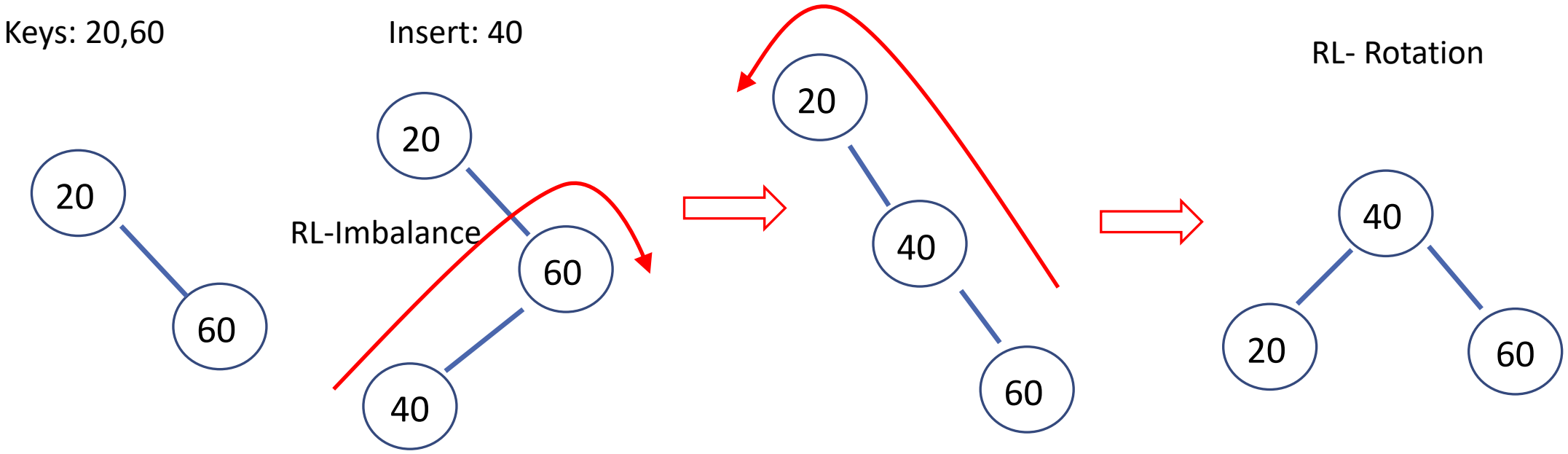
Double Right-Left (RL) Rotation

- The ***double right-left rotation*** (***RL-rotation***) is the mirror image of the double LR-rotation.
- We perform the *L*-rotation of the left subtree of root r followed by the *R*-rotation of the new tree rooted at r.
- It is performed after a new key is inserted into the right subtree of the left child of a tree whose root had the balance of +1 before the insertion.



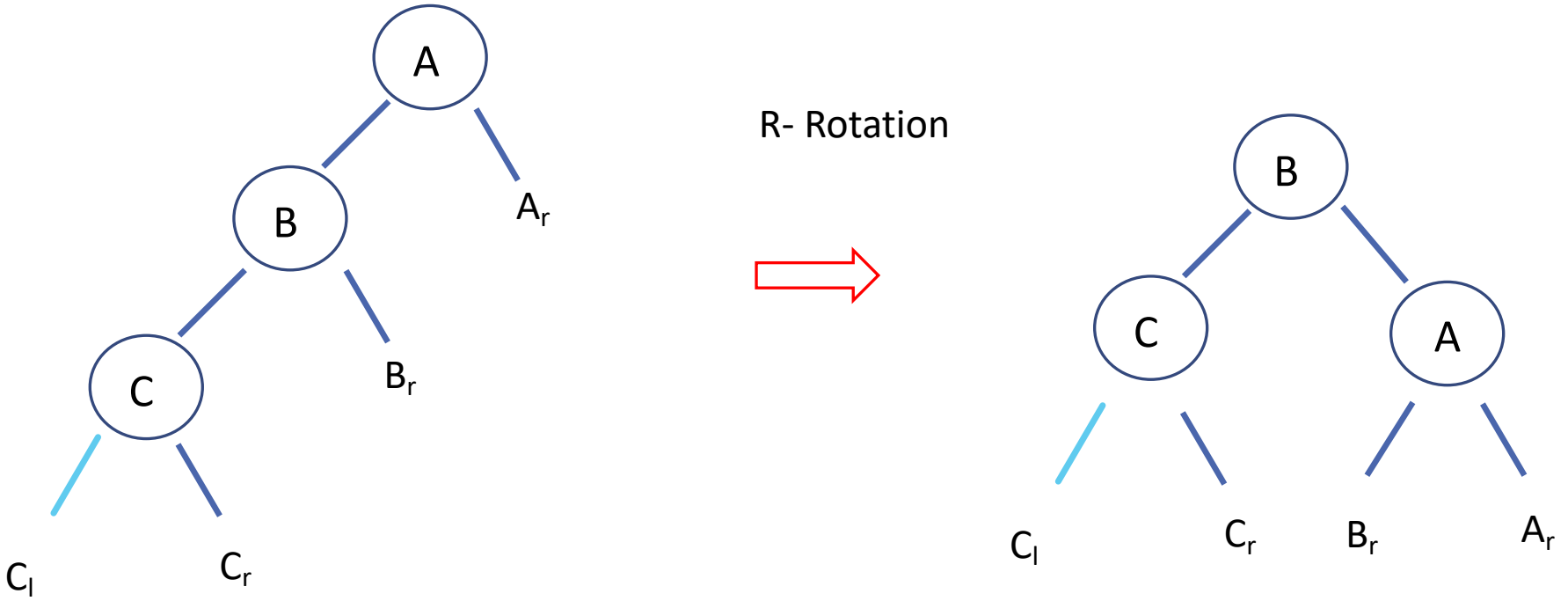
Double RL-rotation

Example: Double RL Rotation



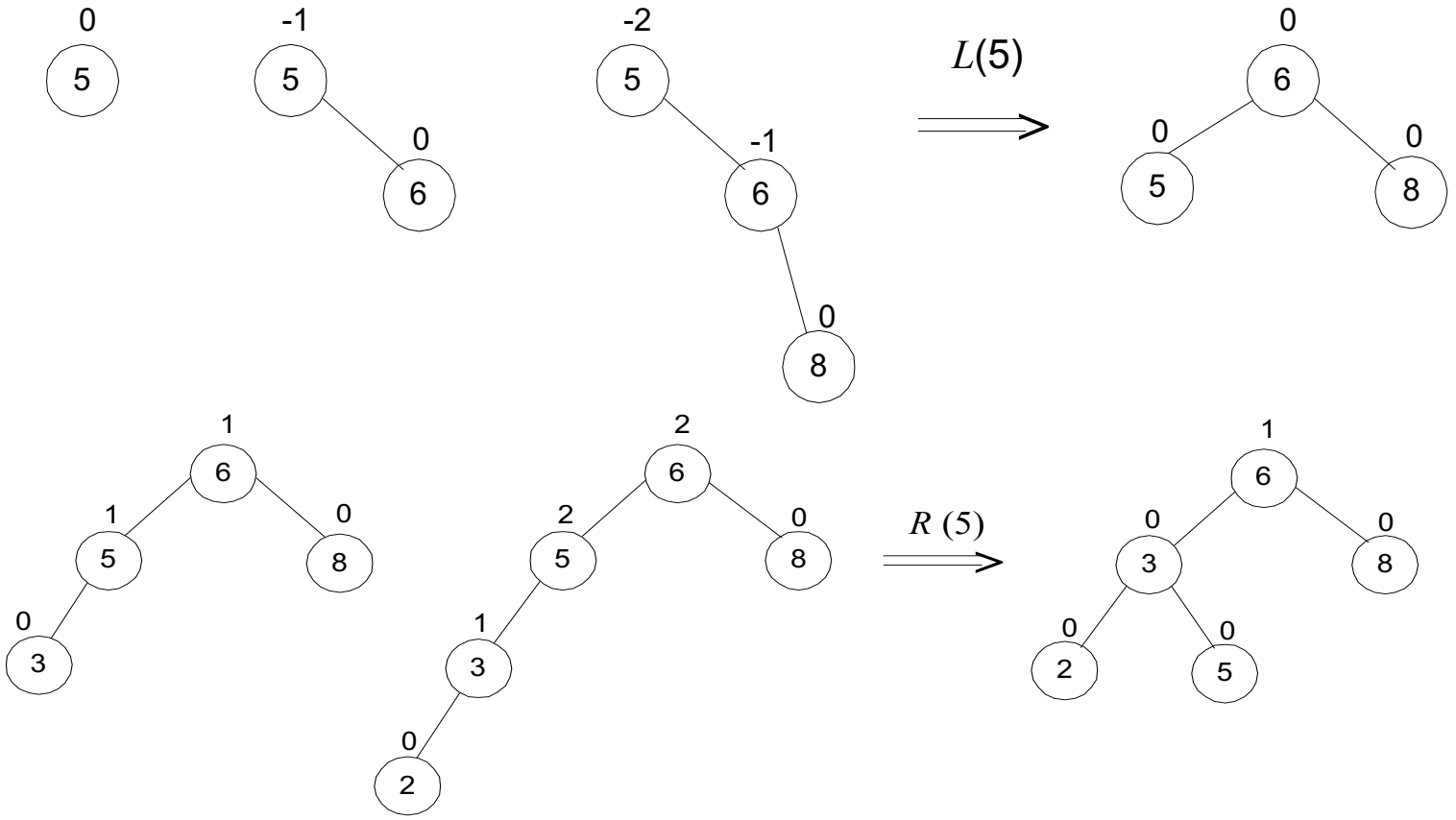
General Case: Single R Rotation

- Rotations guarantee that a resulting tree is balanced, but they should also preserve the basic requirements of a binary search tree.
- And the same relationships among the key values hold, as they must, for the balanced tree after the rotation.



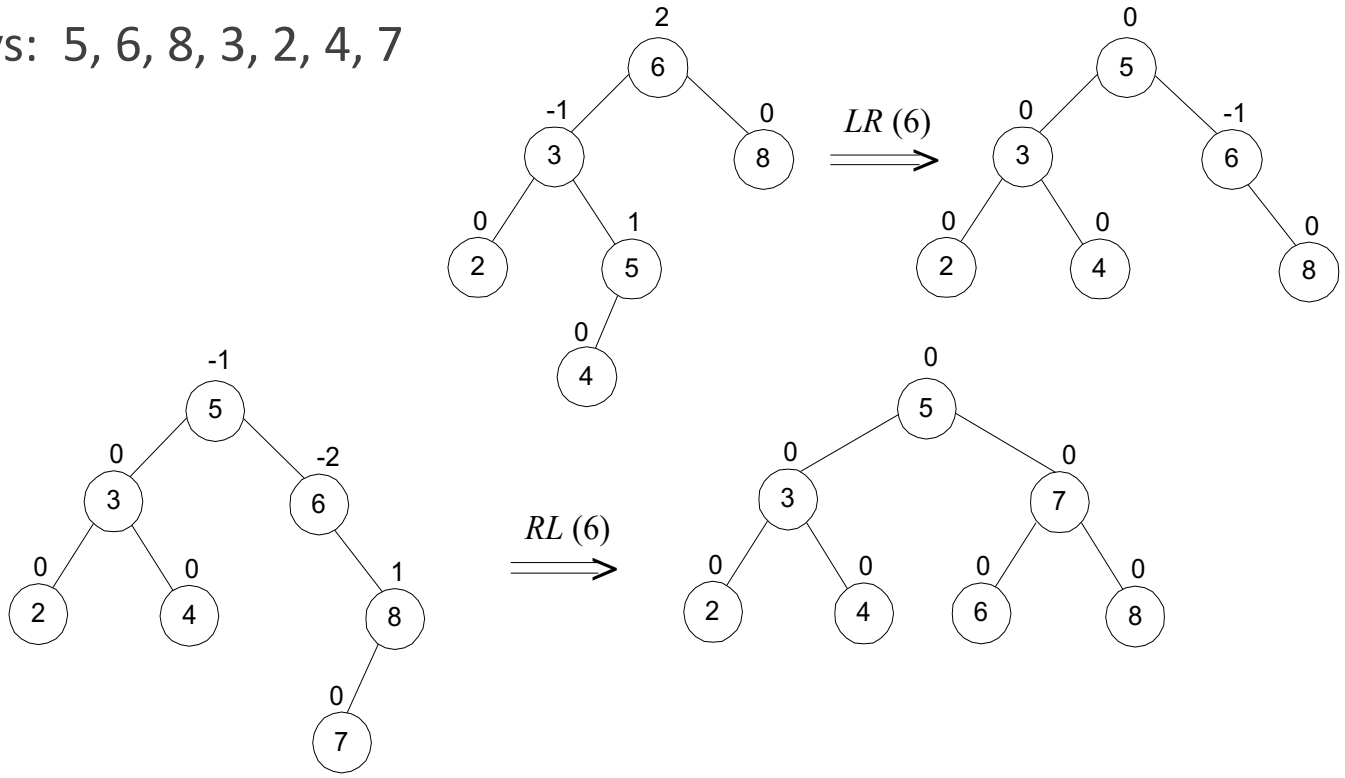
Example 1: AVL tree construction

- Construct an AVL tree for the list 5, 6, 8, 3, 2, 4, 7



Example 1: AVL tree construction (cont.)

- If there are several nodes with the ±2 balance, the rotation is done for the tree rooted at the unbalanced node that is the closest to the newly inserted leaf.
- Keys: 5, 6, 8, 3, 2, 4, 7

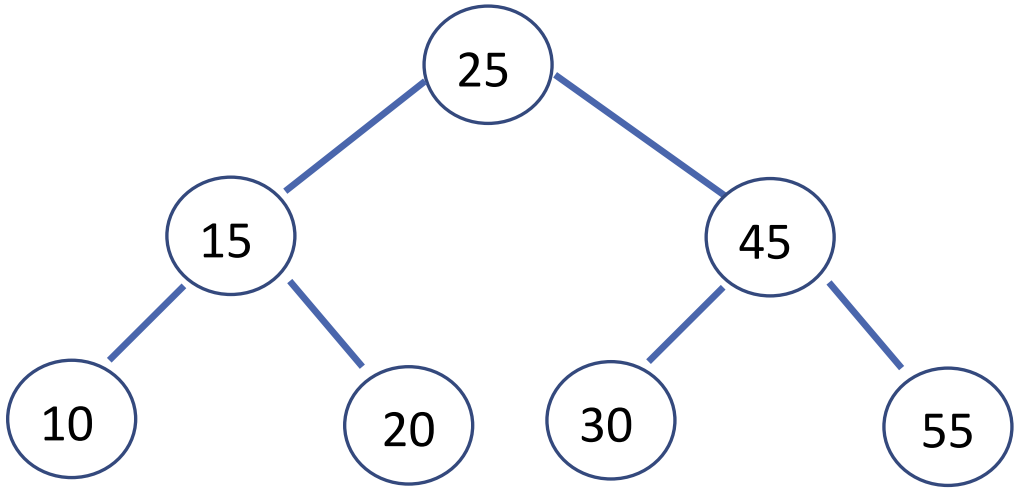


Example 2: AVL tree construction

Construct an AVL tree for the list 45, 15, 10, 25, 30, 20, 55

Example 2: AVL Tree

AVL tree for the list 45, 15, 10, 25, 30, 20, 55



Analysis of AVL trees

- $h \leq 1.4404 \log_2 (n + 2) - 1.3277$
- average height: $1.01 \log_2 n + 0.1$ for large n (found empirically)

- Search and insertion are O(log n)

- Deletion is more complicated but is also O(log n)
- Disadvantages:
  - frequent rotations
  - Complexity
- A similar idea: *red-black trees*

Summary

- Searching Problem
- Binary Search Trees
- AVL Trees

Supportive Materials

- *Introduction to The Design and Analysis of Algorithms (3rd Edition) by Anany Levitin, Pearson.*
- *Introduction to Algorithms*, by T.Cormen, C. Leiserson, R.Rivest and C.Stein, MIT Press, 3rd Edition.